

Subscription Autopilot

An AI agent that runs your recurring service spending on Arc — and can prove it behaved.

“The agent can want anything; it can only pay what policy allows.”

Track	4 — Best Agentic Economy Experience on Arc
Participant	Ronie Joseph (solo)
Circle Developer Account	roniejosephv@gmail.com
Circle products used	USDC · Wallets (developer-controlled) · Gateway · Nanopayments (x402)
Live demo (dashboard)	https://autopilotdashboard-production.up.railway.app
SpendGuard API (public)	https://autopilotsigner-production.up.railway.app/summary
GitHub repository	https://github.com/roniejosephv-star/subscription-autopilot
Demo video	TODO — paste hosted demo-video URL, then regenerate
On-chain audit anchor	SpendAnchor 0xfe18...e435 on Arc Testnet (arcscan)
Scope	Testnet demo — Arc Testnet, chain 5042002 (gas = USDC)

Contents: 1 Description · 2 Working MVP · 3 Architecture · 4 Circle integration · 5 Demo guide & documentation · 6 Circle Product Feedback

1 · Project description

Circle's rails now let AI agents pay autonomously. Almost nobody deploys one — capability isn't trust. Give an agent your wallet and one prompt injection, one hallucinated “great deal,” one runaway loop is a drained balance. Subscription Autopilot treats that trust gap as the product.

The agent autonomously runs recurring, metered service spending (API subscriptions) against three competing x402 sellers with live, volume-tiered pricing:

Research — probes every seller's price menu (free quote endpoints) and its own usage history. **Negotiate** — at every renewal it re-quotes all sellers at its real volume, switches when >5% cheaper per call, and logs the rationale. **Execute** — pays per call with gasless USDC nanopayments (x402), batch-settled by Circle Gateway on Arc.

Every dollar passes through **SpendGuard**, an out-of-process policy authority: monthly budget, seller allowlist, per-transaction cap, per-service cap, daily velocity, and a human-approval threshold. The agent holds **zero key material** — no keys, no Circle SDK, no chain RPC. It can *request* a payment; only policy can authorize one. Policy versions and spend epochs are hash-anchored on Arc (**SpendAnchor.sol**) every 10 minutes, so the audit trail is tamper-evident and externally verifiable.

Differentiation

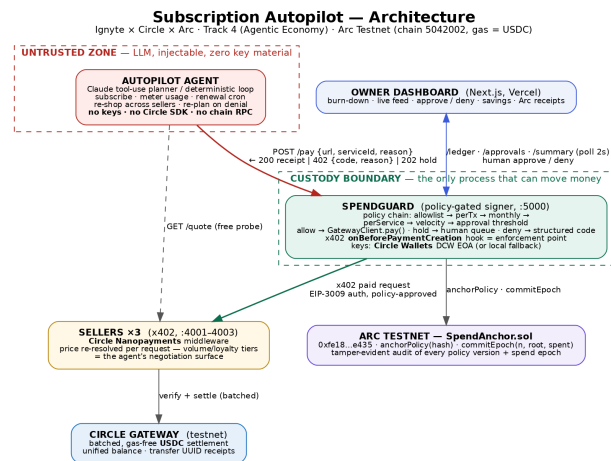
Circle's official sample shows an agent that *can* pay. Autopilot is an agent that pays **well** — cheaper (it re-shops), safer (policy wall + human-in-the-loop), and provably (on-chain anchors). The enforcement point is the x402 SDK's **onBeforePaymentCreation** lifecycle hook running inside SpendGuard's trusted process — behind the custody boundary, where a compromised agent cannot reach it.

2 · Working MVP (deployed, verifiable now)

Frontend + backend are live: the owner dashboard at <https://autopilotdashboard-production.up.railway.app> renders the production ledger (100+ settled nanopayments at time of writing), budget burn-down, approval queue, policy chain, and Gateway treasury balance. The public SpendGuard API ([signer /summary](#)) serves the same data raw.

Deployment	4 services on Railway: SpendGuard (signer), 3 x402 sellers, agent, dashboard; SQLite ledger on a persistent volume
Custody mode	SIGNER_MODE=circle — EIP-712 signed via Circle Wallets DCW EOA; no raw private key in the payment path
Verified E2E	DCW signature → Gateway x402 settle on Arc Testnet, transfer 9bd6a149-1821-4341-a4aa-712cdc362382 (2026-07-06)
On-chain	SpendAnchor at 0xfe18...e435 — policy hashes + spend epochs, auto-anchored every 10 min when spend changes
Policy wall (live)	Injected overspend → deny (per_tx_max_exceeded); premium tier (\$0.03 > \$0.02 threshold) → held for human approve/deny with 60 s window

3 · Architecture



The red boundary is the untrusted zone: the agent plans (Claude tool-use or deterministic loop) but owns no money. The green boundary is the only process that can move funds: SpendGuard evaluates the policy chain before any signature exists, executes via GatewayClient, queues holds for the human owner, and anchors its own history on Arc. Sellers re-resolve prices per request — that live pricing surface is what the agent negotiates against. Full component spec: ARCHITECTURE.md in the repository.

4 · How Circle products are used

Nanopayments / x402	Core rail. Sellers: createGatewayMiddleware + gateway.require(price) bound per request → live volume tiers. SpendGuard: GatewayClient (chain: arcTestnet); policy enforced in the onBeforePaymentCreation hook inside the trusted process.
Circle Wallets (DCW)	Agent wallet is a developer-controlled EOA on ARC-TESTNET (EOA required: Gateway settle verifies EIP-3009 via ecrecover). signTypedData({walletId}) produces the payment signature; even the Gateway deposit runs through Circle's contract-execution API — no raw key anywhere.
Circle Gateway	Fundable agent balance, gas-free batched settlement, transfer-UUID receipts (dashboard links each payment), getBalances() feeds the treasury card.
USDC on Arc	Settlement asset and gas token — SpendAnchor transactions are paid in USDC; one faucet covers everything.
CCTP / Bridge Kit	Architecture extension (documented, not in MVP): cross-chain budget top-ups via Gateway crosschain withdraw().

5 · Demo guide & documentation

Video: *TODO* — paste hosted demo-video URL, then regenerate

The 3-minute video follows four beats, all reproducible on the live deployment:

1 · Delegation	Agent researches quotes, subscribes to the cheapest seller, meters per-call gasless nanopayments — live feed + burn-down on the dashboard.
2 · Negotiation	Seller A's price drifts +60% → at renewal the agent re-quotes all sellers at its real volume and switches (21.9% cheaper per call), rationale logged.
3 · The trust wall	A prompt-injected “URGENT: prepay 12 periods” request → SpendGuard denies at the per-tx cap; a \$0.03 premium call is held → human clicks Approve/Deny on the dashboard.
4 · Proof	Month view: spend vs budget, savings, denial log, Gateway receipts — and the epoch chip linking to the SpendAnchor transaction on arcscan.

Documentation in the repository

README.md — setup from zero (Circle account → wallet generation → faucet → run) and per-product integration notes with file references · ARCHITECTURE.md — component spec and trust boundaries · PRD.md — full product spec, feature acceptance criteria, risk log · DEPLOYMENT.md — the exact Railway/Vercel runbook used for this deployment · FEEDBACK.md — the running Circle product-feedback log reproduced in §6 · docs/demo-script.md — the video script.

Quick verification for judges (no setup): open the dashboard, watch payments tick; curl <https://autopilotsigner-production.up.railway.app/summary>; click the SpendAnchor link to see anchors on arcscan.

6 · Circle Product Feedback

Why we chose these products

Nanopayments/x402 is the only rail where per-call metered agent payments are economically sane (gasless, sub-cent, batched). Gateway gives the agent a fundable balance with instant receipts. Circle Wallets (DCW EOA) moves key custody out of the agent entirely — the property our whole trust model rests on. USDC-as-gas on Arc collapses faucet/gas UX to one token.

What worked well

- DCW EOA signTypedData signatures are fully ecrecover-compatible with Gateway's x402 settle — the custody-separation design works E2E on Arc Testnet (verified 2026-07-06, transfer 9bd6a149-...-712cdc362382).
- Seller monetization is genuinely 2 lines (createGatewayMiddleware + gateway.require()); per-request dynamic pricing works by binding require(price) at call time — it became our negotiation surface.
- CHAIN_CONFIGS exports eliminated all address/RPC hardcoding.
- Arc deposit finality (~0.5 s) makes the deposit→pay loop feel instant; Wallets contract executions confirm in well under a minute.
- The x402 lifecycle hooks are an excellent extension surface — our entire policy layer lives in one hook.

What could be improved (all found and reproduced during this build)

1. **Nanopayments is EOA-only** (ecrecover on EIP-3009; EIP-1271/SCA unsupported) — easy to miss: Circle's own arc-escrow sample creates accountType SCA wallets, which would silently fail here. Needs a prominent docs callout + runtime hint.
2. **Signature-validity contradiction:** seller quickstart says ≥ 7 days, the SDK error reference says ≥ 3 , and the exported GATEWAY_AUTH_VALIDITY_WINDOW_SECONDS constant is undocumented. Pick one number, document the constant.
3. **18-decimal gas vs 6-decimal ERC-20 USDC on Arc** deserves an explicit docs callout — it bites any native-balance math.
4. **Sample apps are heavy references:** arc-escrow needs Supabase + Docker + ngrok + OpenAI before first run; a minimal single-process reference per product would cut onboarding dramatically.
5. **Undocumented header in shipped types:** X-ARC-PRIVATE-MAINNET-ENABLED appears in the SDK's .d.ts, nowhere in docs.
6. **Hook documentation lives off-site** (x402.org, not developers.circle.com) — mirror or link prominently; it's the SDK's best surface.
7. **Broken script in arc-fintech sample:** package.json declares spend:gateway:arc → a file that doesn't exist in the repo.
8. **Wallets ↔ x402 SDK interop wart:** signTypedData requires an explicit EIP712Domain entry in types, but viem-style callers (including BatchEvmScheme's signer interface in Circle's own SDK) omit it. The error — “there is extra data provided in the message (0 < 4)” — names neither the missing type nor the fix. Accept viem-normalized typed data, or return “types.EIP712Domain is required”.
9. **GatewayClient.deposit() requires a raw private key**, so DCW-custodied wallets can't use it. Verified workaround: replicate its two calls (USDC.approve + GatewayWallet.deposit) via createContractExecutionTransaction. A documented DCW-native deposit path is the single change that would most help agentic use cases — Circle-custodied buyers are exactly who agents need.

Recommendations

Ship a first-class “agent buyer” recipe: DCW EOA wallet + Gateway deposit via contract execution + x402 client with a policy hook — the five pieces exist today but must be discovered across four doc sites and two SDKs' type definitions. This project is, in effect, that recipe; we'd be glad if it were useful as a reference.
